

Informe Técnico: Proyecto Unidad 1 - Internet de las Cosas

SmartHome IoT - Sistema de automatización y monitoreo para una casa inteligente

Integrantes: Thomas Lizana, Boris Mejías, Augusto Fuenzalida

Equipo Asignado: Equipo papu

Docente: Ricardo Pérez

Fecha: 03 de Junio, 2026

1. Introducción y descripción del problema

La automatización del hogar representa una de las aplicaciones más consolidadas del Internet de las Cosas (IoT). En la actualidad, una vivienda busca optimizar la seguridad y el monitoreo ambiental ante posibles riesgos domésticos. Para dar solución a esta necesidad, este proyecto plantea el desarrollo de un sistema IoT integral basado en un microcontrolador ESP32 que permite monitorear de forma continua condiciones críticas como la temperatura, niveles de gases peligrosos (GLP) y la calidad del aire en el ambiente (CO2). Además, el sistema incorpora monitoreo acústico para la detección de ruidos anómalos (Micrófono GY-MAX4466) y utiliza una ESP32-CAM en conjunto con un script en Python para procesamiento de visión artificial, identificando personas en tiempo real. Toda la telemetría y el reconocimiento se envían a un broker público y se gestionan mediante Node-RED, consolidando alertas automáticas a través de Telegram y visualización remota en un dashboard centralizado.

2. Objetivo general y objetivos específicos

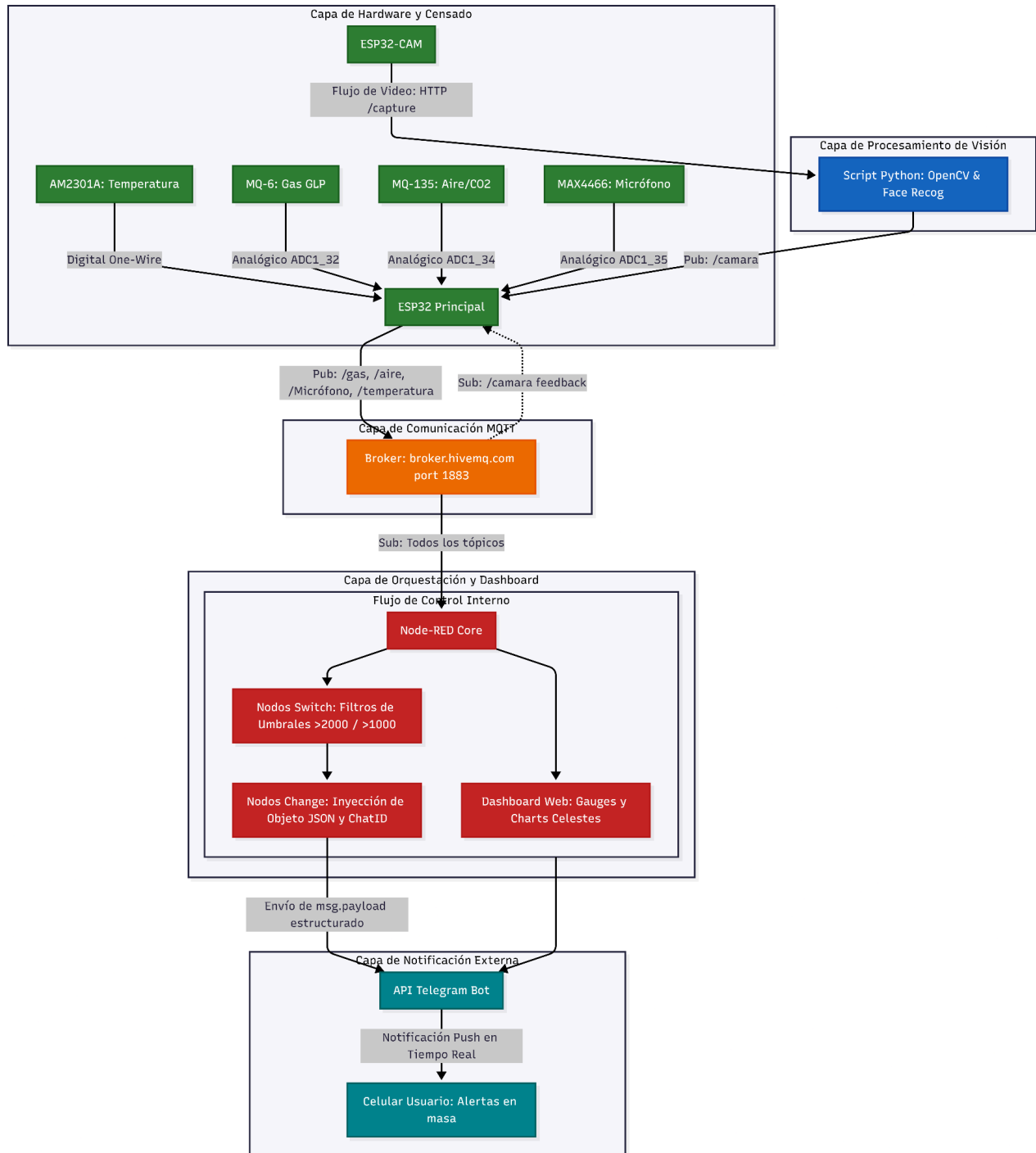
2.1 Objetivo General: Diseñar e implementar un prototipo IoT funcional de monitoreo de seguridad unificado, integrando sensores físicos en un ESP32, procesamiento de visión artificial con OpenCV en Python, comunicación mediante el protocolo MQTT y un centro de control y alertas en Node-RED con integración a Telegram.

2.2 Objetivos Específicos: Conectar y programar los sensores ambientales y acústicos (AM2301A, MQ-6, MQ-135, MAX4466) en una placa ESP32.

- Desplegar una ESP32-CAM para la captura de video en red local y procesar las imágenes mediante un script de Python para reconocimiento de rostros.
- Publicar los datos brutos de los sensores y los resultados de la visión artificial hacia el broker MQTT de HiveMQ utilizando la jerarquía de tópicos smartphone/equipoXX/.
- Construir un dashboard interactivo en Node-RED que incorpore indicadores tipo Gauge y gráficos históricos (Chart) para monitorear las variables en tiempo real.
- Desarrollar estrategias de control mediante nodos lógicos (Switch) para gatillar alertas automáticas de seguridad.
- Vincular el sistema de alertas críticas con un bot de mensajería externa (Telegram) ante

umbrales de riesgo.

3. Diagrama de arquitectura del sistema



4. Descripción de cada sensor utilizado y su rol

Sensor	Modelo/Tipo	Función en el Sistema
Temperatura	AM2301A (DHT21)	Monitoreo ambiental constante de temperatura mediante bus digital One-Wire.
Gases Combustibles	MQ-6	Detección de posibles fugas de Gas Licuado de Petróleo (GLP) e hidrocarburos (ADC).
Calidad del Aire	MQ-135	Medición general de la calidad del aire y acumulación de CO2/humo (ADC).
Micrófono	MAX4466	Monitoreo de amplitud acústica pico a pico para detectar ruidos fuertes o golpes en el entorno (ADC).
Cámara IP	ESP32-CAM	Captura de video en tiempo real enviada a un servidor Python para la identificación de personas por reconocimiento facial.

5. Estructura de tópicos MQTT y ejemplos JSON

Jerarquía utilizada:

smartphone/equipoXX/ (Sustituyendo *smarthome* según la adaptación técnica del equipo).

Tópicos de Sensores (Carga útil en texto plano numérico):

- smartphone/equipoXX/gas → Ejemplo de payload: 2327
- smartphone/equipoXX/aire → Ejemplo de payload: 1459
- smartphone/equipoXX/Micrófono → Ejemplo de payload: 234
- smartphone/equipoXX/temperatura → Ejemplo de payload: 24.5
- smarthome/equipoXX/camara Ejemplo de payload: "Augusto" (Enviado desde Python)

Ejemplo de estructura interna JSON generada en Node-RED para la API de Telegram:

JSON

```
{  
  "chatId": 6701169878,  
  "type": "message",  
  "content": "ALERTA, MUCHO GAS (MAYOR A 2000)"  
}
```

6. Descripción del dashboard y capturas de pantalla

El dashboard en Node-RED centraliza las mediciones en un único grupo visual ("EstacionUnificada"):

- **Gauges (Medidores de aguja):** Tres medidores independientes muestran en tiempo real las lecturas de Gas, Calidad de Aire y Temperatura (escalada de 0 a 50 °C).
- **Chart (Gráfico Lineal):** Un visualizador dinámico grafica la de amplitud de sonido leídos por el micrófono, permitiendo ver el comportamiento acústico en una ventana de tiempo.
- **Text (Caja de estado):** Un indicador muestra el último string recibido del reconocimiento facial ("Última persona detectada").

7. Estrategias de control implementadas

El control automático se realiza en Node-RED filtrando datos en bruto de los sensores:

- **Regla 1 (Alerta de Fuga de Gas):** Si la lectura del sensor MQ-6 supera el valor de 2000 (umbral de peligro), se genera automáticamente un payload con el formato requerido por la API de Telegram y se envía un mensaje de emergencia. Esta regla también aplica para el sensor MQ-135, que es el sensor de calidad de aire.
- **Regla 2 (Alerta de Ruido/Intrusión):** Si la amplitud medida por el MAX4466 es mayor a 1000 (indicativo de un golpe fuerte o ruido anormal), se activa el Switch y se despacha

una notificación de ruido a Telegram.

- **Regla 3 (Alerta de temperatura):** Si la amplitud medida por el Sensor de Temperatura es mayor a 20 °C, se activa el Switch y se despacha una notificación de "Hace Calor".

8. Descripción de notificaciones y registro histórico

Canal de notificación: Bot oficial de Telegram integrado mediante los nodos `node-red-contrib-telegrambot`. Los mensajes son dirigidos unívocamente al Chat ID privado configurado.

Método de registro: Visualización de tendencias temporales en tiempo real en la memoria volátil del nodo "Chart" de Node-RED, y visualización de logs de estado en la terminal del microcontrolador.

9. Problemas encontrados y soluciones aplicadas

Durante el desarrollo e integración, se abordaron diversos desafíos técnicos:

1. **Error de formato de mensajes MQTT a Telegram:** Se presentó el error en depuración `"Cannot set property of non-object type: payload.type"`. Esto ocurrió al intentar asignar sub-atributos de Telegram a un payload que llegaba como número bruto desde el ESP32. **Solución:** Se reestructuró el nodo *Change* agregando una regla previa que establece `msg.payload` explícitamente como un objeto JSON vacío `{}` antes de asignarle las propiedades de alerta.
2. **Bad Request de la API de Telegram:** El servidor de mensajería rechazaba las alertas indicando `"chat_id is empty"`. **Solución:** Al depurar el objeto JSON, se detectó un error tipográfico en la asignación de Node-RED (`payload, chatId` con coma). Se reemplazó por la notación de punto correcta (`payload.chatId`), habilitando la comunicación exitosa con el bot.
3. **Lecturas gráficas no compatibles (Gauge Error):** Al integrar la temperatura AM2301A, el nodo arrojó `"invalid property expression"`. **Solución:** Se purgaron variables residuales del campo *Value* (que debía ser estrictamente `payload`) y se migró la sintaxis de la unidad al apartado de visualización (*Labelling -> Units*).

10. Conclusiones

El proyecto demostró la viabilidad de construir un sistema de monitoreo IoT híbrido robusto y funcional a bajo costo. Se logró integrar con éxito la telemetría en C++ (ESP32) con el procesamiento de visión artificial en Python y la orquestación lógica en Node-RED (JavaScript). La utilización de MQTT como puente de comunicaciones probó ser eficiente para manejar flujos de datos asíncronos y desacoplados. Asimismo, la implementación de un bot de Telegram certificó que el prototipo no solo sirve como herramienta de observación local, sino como un sistema proactivo capaz de notificar al usuario de forma remota y en tiempo real frente a

eventos críticos de seguridad.

11. Referencias y Bibliografía

Librerías de Desarrollo y Hardware (Microcontroladores)

- **Espressif Systems.** (s.f.). *Arduino core for the ESP32*. Documentación oficial del núcleo de desarrollo. Recuperado de <https://docs.espressif.com/projects/arduino-esp32/>
- **O'Leary, N.** (s.f.). *PubSubClient*. Librería de cliente MQTT para entornos Arduino. Recuperado de <https://pubsubclient.knolleary.net/>
- **Blanchon, B.** (s.f.). *ArduinoJson*. Librería de serialización y deserialización de datos JSON para C++ en sistemas embebidos. Recuperado de <https://arduinojson.org/>
- **Adafruit Industries.** (s.f.). *DHT Sensor Library*. Controladores y utilidades para sensores de temperatura y humedad de la familia DHT/AM2301. Recuperado de <https://github.com/adafruit/DHT-sensor-library>

Plataforma de Orquestación IoT (Node-RED)

- **OpenJS Foundation.** (s.f.). *Node-RED: Low-code programming for event-driven applications*. Documentación oficial de la plataforma. Recuperado de <https://nodered.org/docs/>
- **Node-RED Dashboard.** (s.f.). *node-red-dashboard*. Repositorio del módulo de componentes gráficos de interfaz de usuario. Recuperado de <https://flows.nodered.org/node/node-red-dashboard>
- **Node-RED Telegram Bot.** (s.f.). *node-red-contrib-telegrambot*. Paquete de nodos para la implementación y consumo de la API de Telegram. Recuperado de <https://flows.nodered.org/node/node-red-contrib-telegrambot>

Protocolos de Red y Servicios Externos

- **HiveMQ.** (s.f.). *HiveMQ Public MQTT Broker*. Especificaciones y acceso al servidor público de mensajería IoT. Recuperado de <https://www.hivemq.com/public-mqtt-broker/>
- **Telegram.** (s.f.). *Telegram Bot API*. Documentación técnica oficial provista por Telegram para desarrolladores. Recuperado de <https://core.telegram.org/bots/api>